

Báo cáo bài tập ngày 16/6/2026

Họ và tên:

Nguyễn Kim Tùng Quân – 20235407 – Chat_server

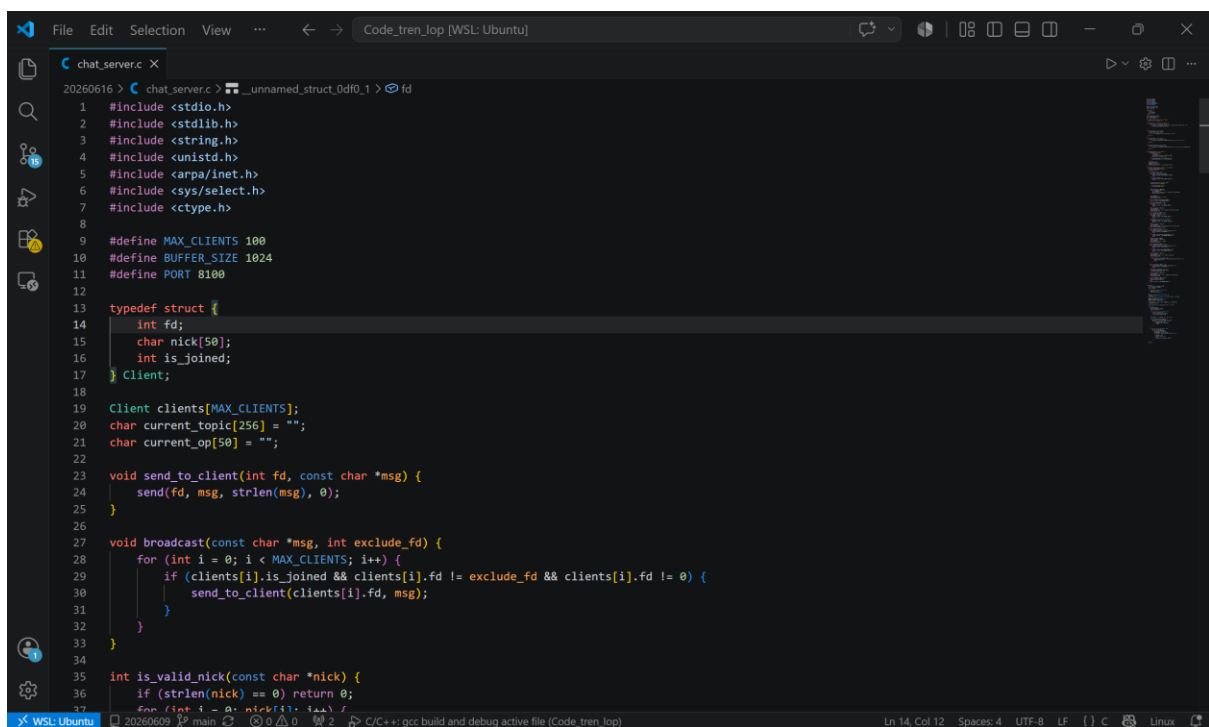
Trần Anh Phú – 20235397 – Chat_client

Bài tập: Sinh viên lập trình ứng dụng Chat Server / Client theo giao thức được mô tả trong slide.

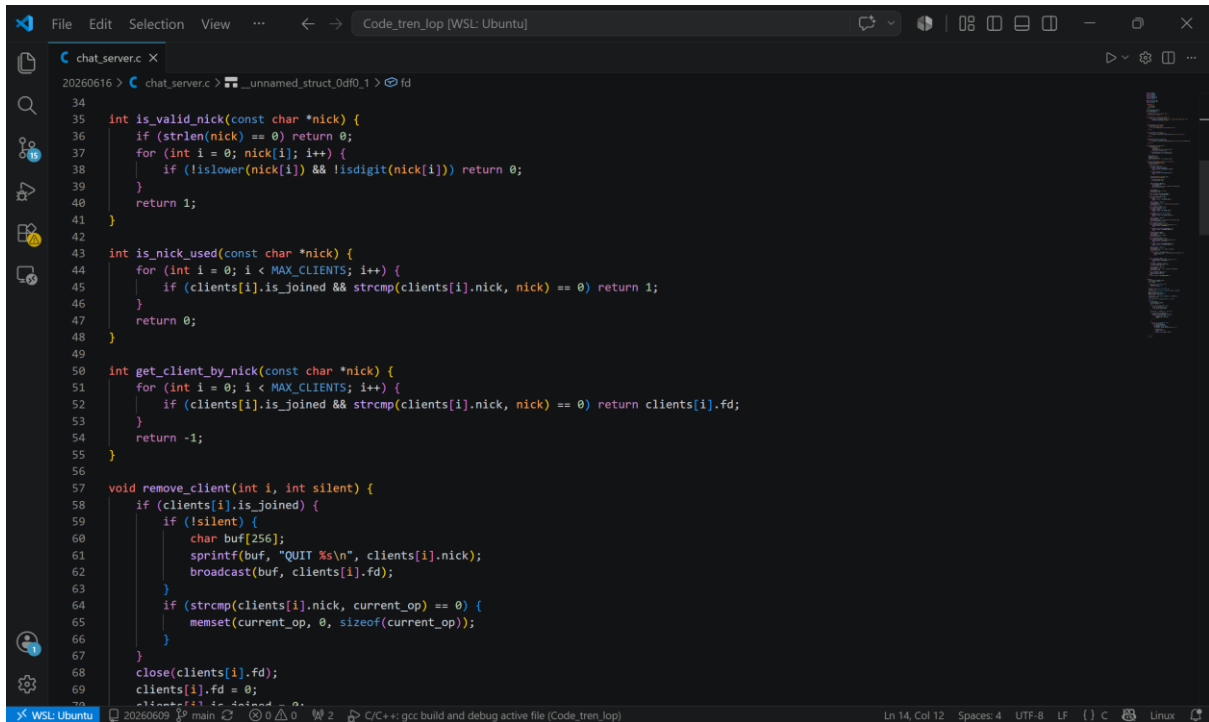
Mỗi nhóm 2 sinh viên:

- + Một người lập trình ứng dụng chat server
- + Một người lập trình ứng dụng chat client (sử dụng giao diện tùy ý)
- * Đối với ứng dụng server
 - + Tải và chạy file kiểm thử tại địa chỉ http://lebavui.io.vn/chat_server_test
 - + Cấp quyền thực thi cho file kiểm thử
 - + Chạy file kiểm thử và nộp ảnh chụp kết quả

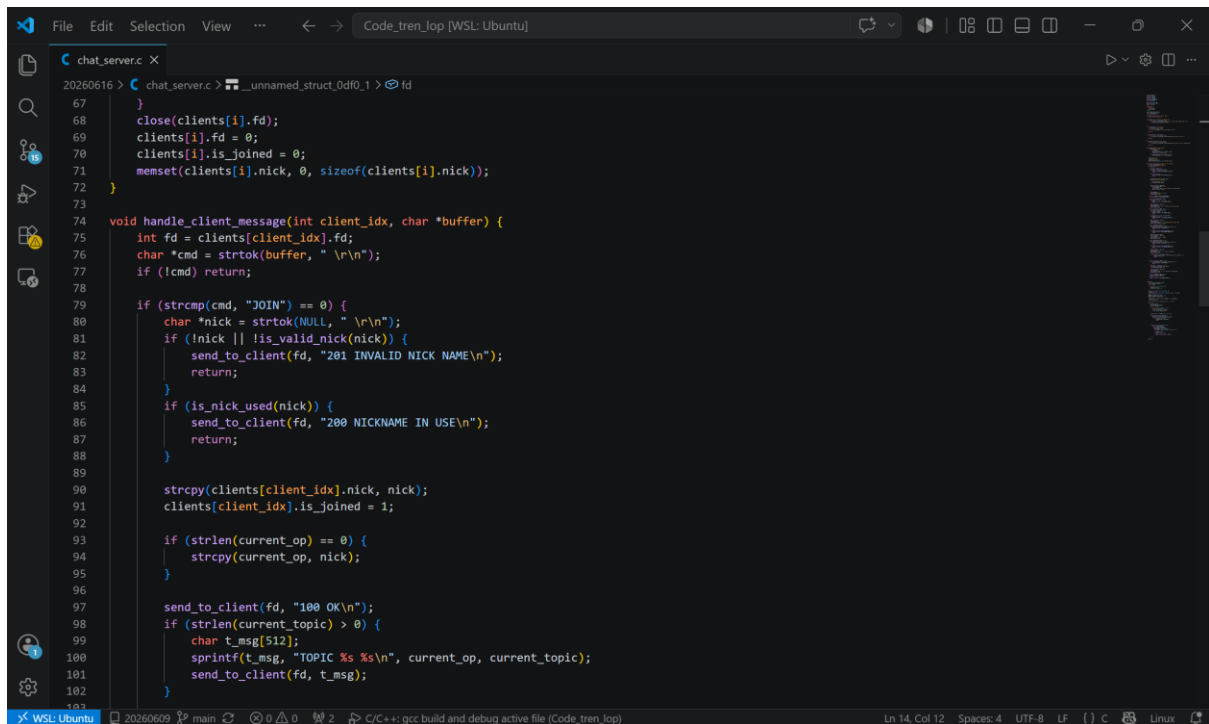
Code Server:



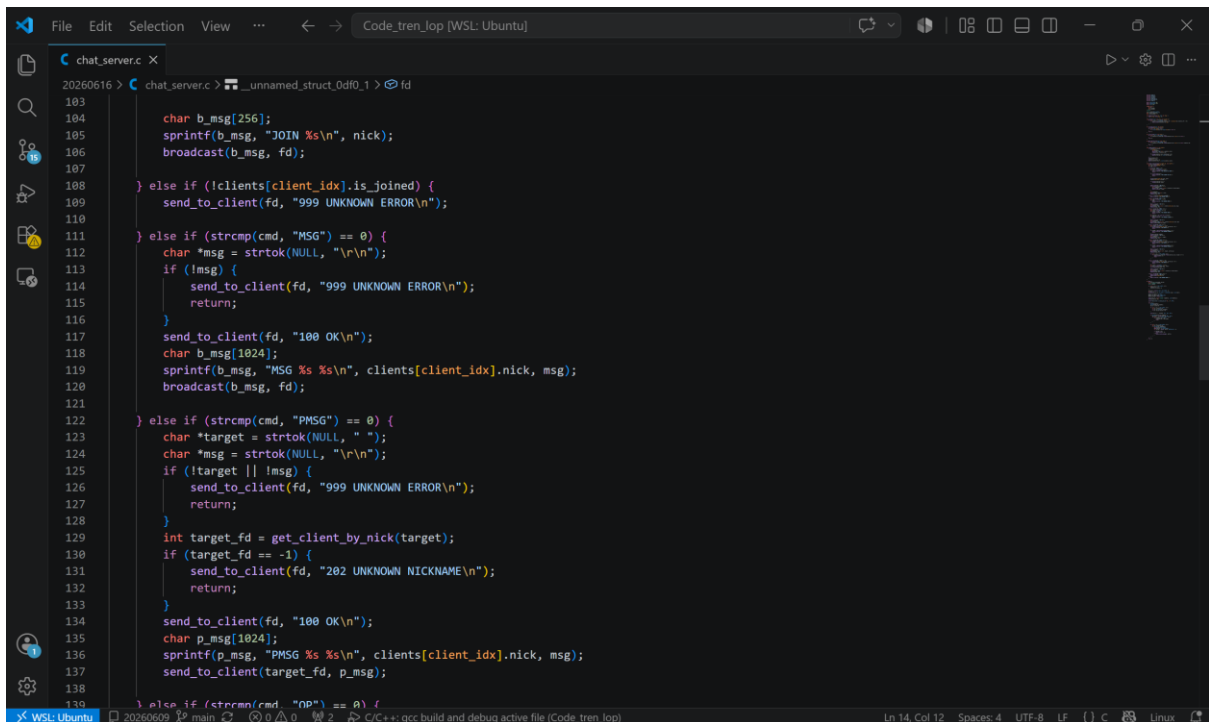
```
1 #include <stdio.h>
2 #include <stdlib.h>
3 #include <string.h>
4 #include <unistd.h>
5 #include <arpa/inet.h>
6 #include <sys/select.h>
7 #include <ctype.h>
8
9 #define MAX_CLIENTS 100
10 #define BUFFER_SIZE 1024
11 #define PORT 8100
12
13 typedef struct {
14     int fd;
15     char nick[50];
16     int is_joined;
17 } Client;
18
19 Client clients[MAX_CLIENTS];
20 char current_topic[256] = "";
21 char current_op[50] = "";
22
23 void send_to_client(int fd, const char *msg) {
24     send(fd, msg, strlen(msg), 0);
25 }
26
27 void broadcast(const char *msg, int exclude_fd) {
28     for (int i = 0; i < MAX_CLIENTS; i++) {
29         if (clients[i].is_joined && clients[i].fd != exclude_fd && clients[i].fd != 0) {
30             send_to_client(clients[i].fd, msg);
31         }
32     }
33 }
34
35 int is_valid_nick(const char *nick) {
36     if (strlen(nick) == 0) return 0;
37     for (int i = 0; i < strlen(nick); i++) {
38         if (!isalnum(nick[i])) return 0;
39     }
40     return 1;
41 }
```



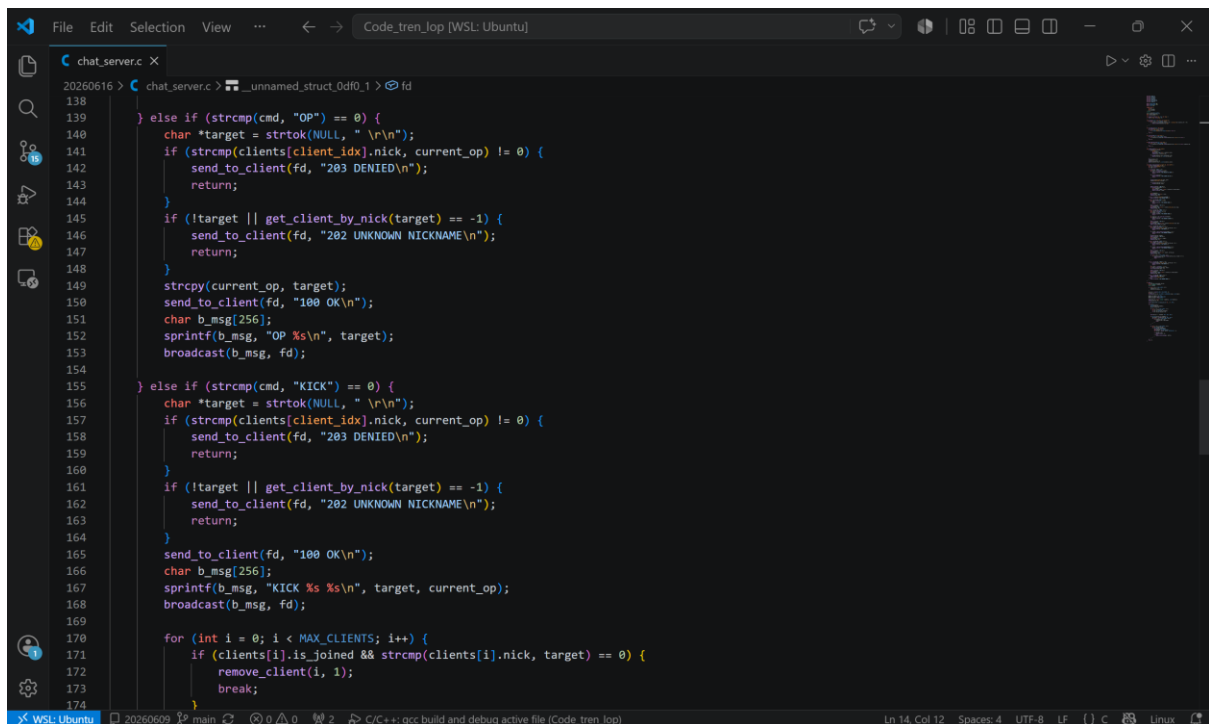
```
File Edit Selection View ... Code_tren_lop [WSL: Ubuntu]
chat_server.c X
20260616 > chat_server.c > _unnamed_struct_0df0_1 > fd
34
35 int is_valid_nick(const char *nick) {
36     if (strlen(nick) == 0) return 0;
37     for (int i = 0; nick[i]; i++) {
38         if (!islower(nick[i]) && !isdigit(nick[i])) return 0;
39     }
40     return 1;
41 }
42
43 int is_nick_used(const char *nick) {
44     for (int i = 0; i < MAX_CLIENTS; i++) {
45         if (clients[i].is_joined && strcmp(clients[i].nick, nick) == 0) return 1;
46     }
47     return 0;
48 }
49
50 int get_client_by_nick(const char *nick) {
51     for (int i = 0; i < MAX_CLIENTS; i++) {
52         if (clients[i].is_joined && strcmp(clients[i].nick, nick) == 0) return clients[i].fd;
53     }
54     return -1;
55 }
56
57 void remove_client(int i, int silent) {
58     if (clients[i].is_joined) {
59         if (!silent) {
60             char buf[256];
61             sprintf(buf, "QUIT %s\n", clients[i].nick);
62             broadcast(buf, clients[i].fd);
63         }
64         if (strcmp(clients[i].nick, current_op) == 0) {
65             memset(current_op, 0, sizeof(current_op));
66         }
67     }
68     close(clients[i].fd);
69     clients[i].fd = 0;
70     clients[i].is_joined = 0;
71     memset(clients[i].nick, 0, sizeof(clients[i].nick));
72 }
73
74 void handle_client_message(int client_idx, char *buffer) {
75     int fd = clients[client_idx].fd;
76     char *cmd = strtok(buffer, " \r\n");
77     if (!cmd) return;
78
79     if (strcmp(cmd, "JOIN") == 0) {
80         char *nick = strtok(NULL, " \r\n");
81         if (!nick || !is_valid_nick(nick)) {
82             send_to_client(fd, "201 INVALID NICK NAME\n");
83             return;
84         }
85         if (is_nick_used(nick)) {
86             send_to_client(fd, "200 NICKNAME IN USE\n");
87             return;
88         }
89         strcpy(clients[client_idx].nick, nick);
90         clients[client_idx].is_joined = 1;
91
92         if (strlen(current_op) == 0) {
93             strcpy(current_op, nick);
94         }
95
96         send_to_client(fd, "100 OK\n");
97         if (strlen(current_topic) > 0) {
98             char t_msg[512];
99             sprintf(t_msg, "TOPIC %s %s\n", current_op, current_topic);
100             send_to_client(fd, t_msg);
101         }
102     }
103 }
```



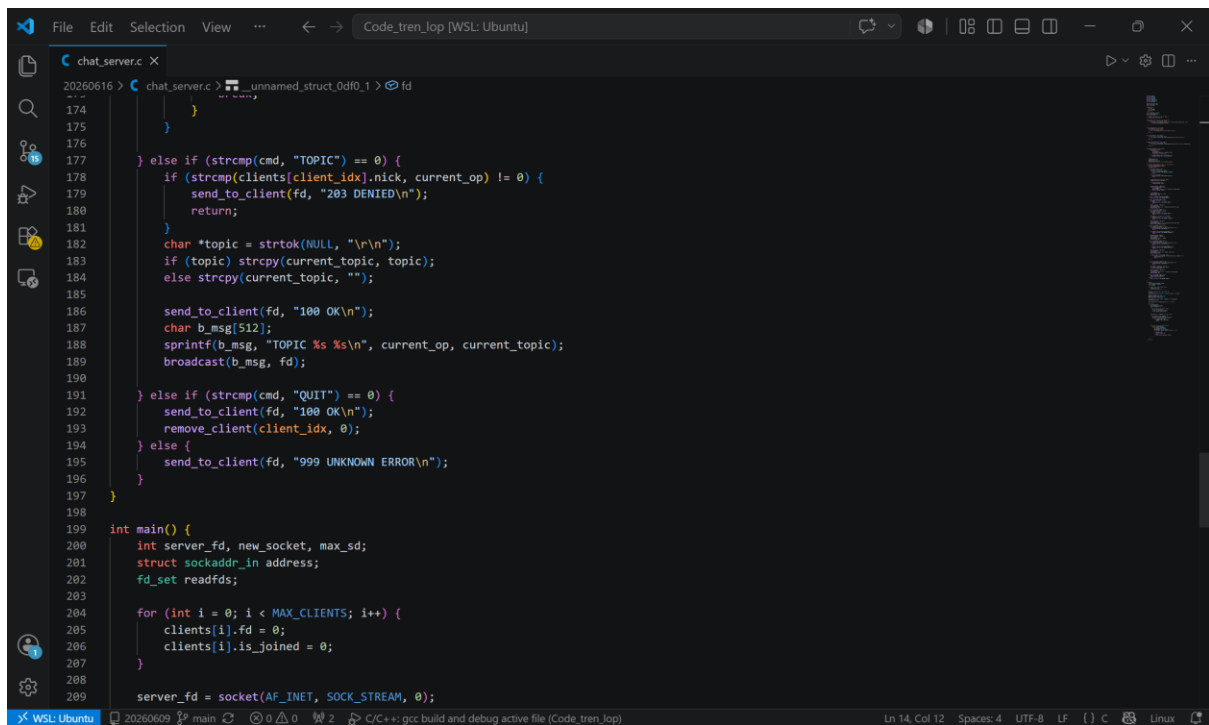
```
File Edit Selection View ... Code_tren_lop [WSL: Ubuntu]
chat_server.c X
20260616 > chat_server.c > _unnamed_struct_0df0_1 > fd
67
68 close(clients[i].fd);
69 clients[i].fd = 0;
70 clients[i].is_joined = 0;
71 memset(clients[i].nick, 0, sizeof(clients[i].nick));
72 }
73
74 void handle_client_message(int client_idx, char *buffer) {
75     int fd = clients[client_idx].fd;
76     char *cmd = strtok(buffer, " \r\n");
77     if (!cmd) return;
78
79     if (strcmp(cmd, "JOIN") == 0) {
80         char *nick = strtok(NULL, " \r\n");
81         if (!nick || !is_valid_nick(nick)) {
82             send_to_client(fd, "201 INVALID NICK NAME\n");
83             return;
84         }
85         if (is_nick_used(nick)) {
86             send_to_client(fd, "200 NICKNAME IN USE\n");
87             return;
88         }
89         strcpy(clients[client_idx].nick, nick);
90         clients[client_idx].is_joined = 1;
91
92         if (strlen(current_op) == 0) {
93             strcpy(current_op, nick);
94         }
95
96         send_to_client(fd, "100 OK\n");
97         if (strlen(current_topic) > 0) {
98             char t_msg[512];
99             sprintf(t_msg, "TOPIC %s %s\n", current_op, current_topic);
100             send_to_client(fd, t_msg);
101         }
102     }
103 }
```



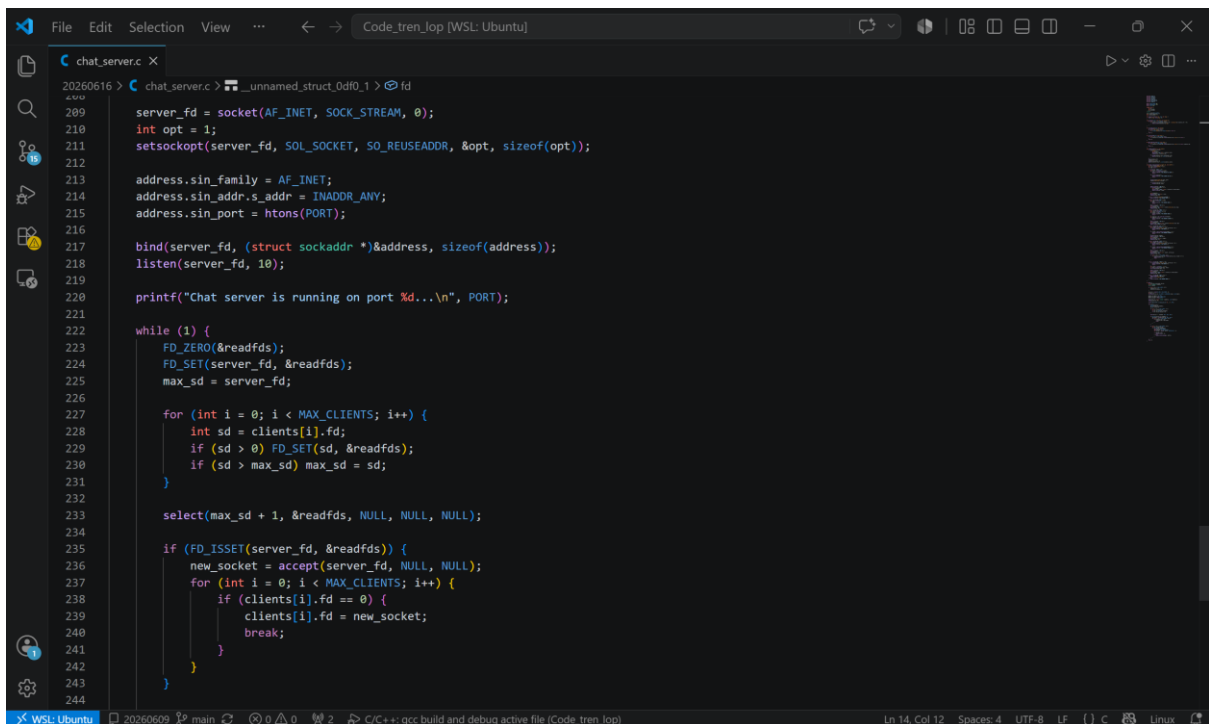
```
File Edit Selection View ... Code_tren_lop [WSL: Ubuntu]
chat_server.c X
20260616 > chat_server.c > _unnamed_struct_0d0_1 > fd
103
104     char b_msg[256];
105     sprintf(b_msg, "JOIN %s\n", nick);
106     broadcast(b_msg, fd);
107
108 } else if (!clients[client_idx].is_joined) {
109     send_to_client(fd, "999 UNKNOWN ERROR\n");
110 }
111 } else if (strcmp(cmd, "MSG") == 0) {
112     char *msg = strtok(NULL, "\n");
113     if (!msg) {
114         send_to_client(fd, "999 UNKNOWN ERROR\n");
115         return;
116     }
117     send_to_client(fd, "100 OK\n");
118     char b_msg[1024];
119     sprintf(b_msg, "MSG %s %s\n", clients[client_idx].nick, msg);
120     broadcast(b_msg, fd);
121
122 } else if (strcmp(cmd, "PMSG") == 0) {
123     char *target = strtok(NULL, " ");
124     char *msg = strtok(NULL, "\n");
125     if (!target || !msg) {
126         send_to_client(fd, "999 UNKNOWN ERROR\n");
127         return;
128     }
129     int target_fd = get_client_by_nick(target);
130     if (target_fd == -1) {
131         send_to_client(fd, "202 UNKNOWN NICKNAME\n");
132         return;
133     }
134     send_to_client(fd, "100 OK\n");
135     char p_msg[1024];
136     sprintf(p_msg, "PMSG %s %s\n", clients[client_idx].nick, msg);
137     send_to_client(target_fd, p_msg);
138
139 } else if (strcmp(cmd, "OP") == 0) {
140
141     char *target = strtok(NULL, "\n");
142     if (strcmp(clients[client_idx].nick, current_op) != 0) {
143         send_to_client(fd, "203 DENIED\n");
144         return;
145     }
146     if (!target || get_client_by_nick(target) == -1) {
147         send_to_client(fd, "202 UNKNOWN NICKNAME\n");
148         return;
149     }
150     strcpy(current_op, target);
151     send_to_client(fd, "100 OK\n");
152     char b_msg[256];
153     sprintf(b_msg, "OP %s\n", target);
154     broadcast(b_msg, fd);
155
156 } else if (strcmp(cmd, "KICK") == 0) {
157     char *target = strtok(NULL, "\n");
158     if (strcmp(clients[client_idx].nick, current_op) != 0) {
159         send_to_client(fd, "203 DENIED\n");
160         return;
161     }
162     if (!target || get_client_by_nick(target) == -1) {
163         send_to_client(fd, "202 UNKNOWN NICKNAME\n");
164         return;
165     }
166     send_to_client(fd, "100 OK\n");
167     char b_msg[256];
168     sprintf(b_msg, "KICK %s %s\n", target, current_op);
169     broadcast(b_msg, fd);
170
171     for (int i = 0; i < MAX_CLIENTS; i++) {
172         if (clients[i].is_joined && strcmp(clients[i].nick, target) == 0) {
173             remove_client(i, 1);
174             break;
175         }
176     }
177
178 } else if (strcmp(cmd, "NP") == 0) {
179
180     char *target = strtok(NULL, "\n");
181     if (strcmp(clients[client_idx].nick, current_op) != 0) {
182         send_to_client(fd, "203 DENIED\n");
183         return;
184     }
185     if (!target || get_client_by_nick(target) == -1) {
186         send_to_client(fd, "202 UNKNOWN NICKNAME\n");
187         return;
188     }
189     send_to_client(fd, "100 OK\n");
190     char n_msg[1024];
191     sprintf(n_msg, "NP %s %s\n", target, current_op);
192     broadcast(n_msg, fd);
193
194 } else if (strcmp(cmd, "QUIT") == 0) {
195     send_to_client(fd, "100 OK\n");
196     remove_client(client_idx, 1);
197 }
198
199 return 0;
200 }
```



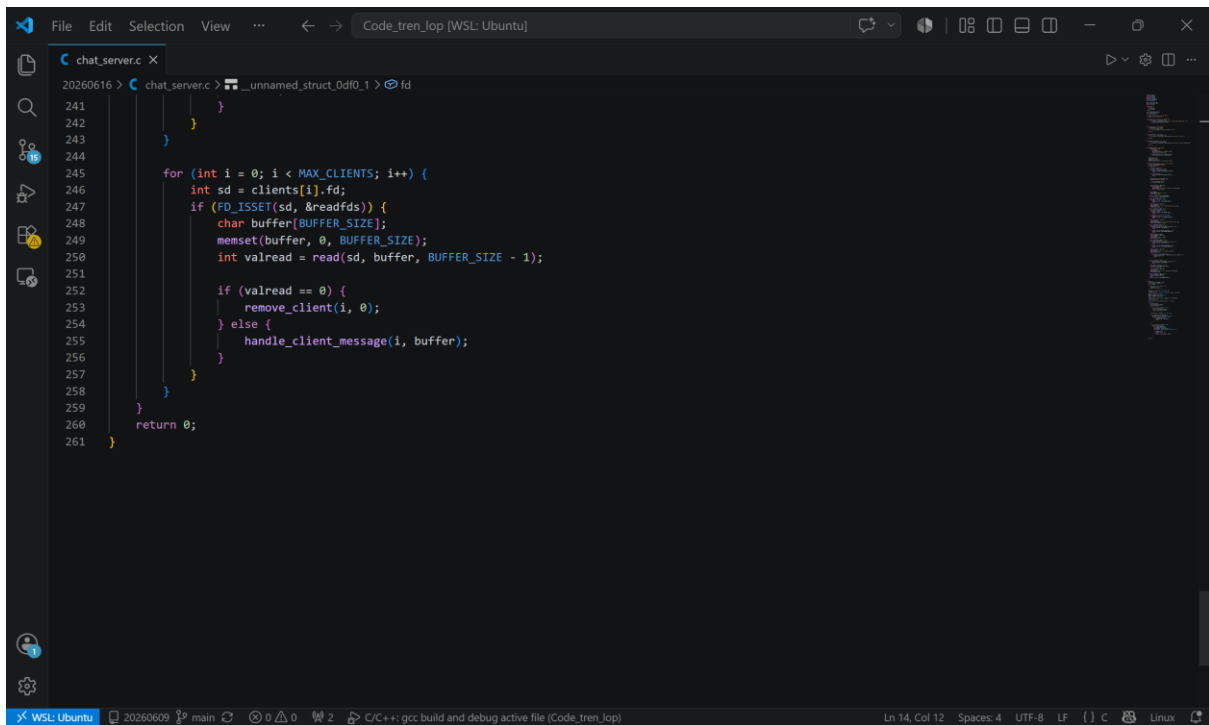
```
File Edit Selection View ... Code_tren_lop [WSL: Ubuntu]
chat_server.c X
20260616 > chat_server.c > _unnamed_struct_0d0_1 > fd
138
139 } else if (strcmp(cmd, "OP") == 0) {
140     char *target = strtok(NULL, "\n");
141     if (strcmp(clients[client_idx].nick, current_op) != 0) {
142         send_to_client(fd, "203 DENIED\n");
143         return;
144     }
145     if (!target || get_client_by_nick(target) == -1) {
146         send_to_client(fd, "202 UNKNOWN NICKNAME\n");
147         return;
148     }
149     strcpy(current_op, target);
150     send_to_client(fd, "100 OK\n");
151     char b_msg[256];
152     sprintf(b_msg, "OP %s\n", target);
153     broadcast(b_msg, fd);
154
155 } else if (strcmp(cmd, "KICK") == 0) {
156     char *target = strtok(NULL, "\n");
157     if (strcmp(clients[client_idx].nick, current_op) != 0) {
158         send_to_client(fd, "203 DENIED\n");
159         return;
160     }
161     if (!target || get_client_by_nick(target) == -1) {
162         send_to_client(fd, "202 UNKNOWN NICKNAME\n");
163         return;
164     }
165     send_to_client(fd, "100 OK\n");
166     char b_msg[256];
167     sprintf(b_msg, "KICK %s %s\n", target, current_op);
168     broadcast(b_msg, fd);
169
170     for (int i = 0; i < MAX_CLIENTS; i++) {
171         if (clients[i].is_joined && strcmp(clients[i].nick, target) == 0) {
172             remove_client(i, 1);
173             break;
174         }
175     }
176
177 } else if (strcmp(cmd, "NP") == 0) {
178
179     char *target = strtok(NULL, "\n");
180     if (strcmp(clients[client_idx].nick, current_op) != 0) {
181         send_to_client(fd, "203 DENIED\n");
182         return;
183     }
184     if (!target || get_client_by_nick(target) == -1) {
185         send_to_client(fd, "202 UNKNOWN NICKNAME\n");
186         return;
187     }
188     send_to_client(fd, "100 OK\n");
189     char n_msg[1024];
190     sprintf(n_msg, "NP %s %s\n", target, current_op);
191     broadcast(n_msg, fd);
192
193 } else if (strcmp(cmd, "QUIT") == 0) {
194     send_to_client(fd, "100 OK\n");
195     remove_client(client_idx, 1);
196 }
197
198 return 0;
199 }
```



```
File Edit Selection View ... Code_tren_lop [WSL: Ubuntu]
chat_server.c X
20260616 > chat_server.c > _unnamed_struct_0df0_1 > fd
174
175
176
177 } else if (strcmp(cmd, "TOPIC") == 0) {
178     if (strcmp(clients[client_idx].nick, current_op) != 0) {
179         send_to_client(fd, "203 DENIED\n");
180         return;
181     }
182     char *topic = strtok(NULL, "\n\n");
183     if (topic) strcpy(current_topic, topic);
184     else strcpy(current_topic, "");
185
186     send_to_client(fd, "100 OK\n");
187     char b_msg[512];
188     sprintf(b_msg, "TOPIC %s %s\n", current_op, current_topic);
189     broadcast(b_msg, fd);
190
191 } else if (strcmp(cmd, "QUIT") == 0) {
192     send_to_client(fd, "100 OK\n");
193     remove_client(client_idx, 0);
194 } else {
195     send_to_client(fd, "999 UNKNOWN ERROR\n");
196 }
197
198
199 int main() {
200     int server_fd, new_socket, max_sd;
201     struct sockaddr_in address;
202     fd_set readfds;
203
204     for (int i = 0; i < MAX_CLIENTS; i++) {
205         clients[i].fd = 0;
206         clients[i].is_joined = 0;
207     }
208
209     server_fd = socket(AF_INET, SOCK_STREAM, 0);
```

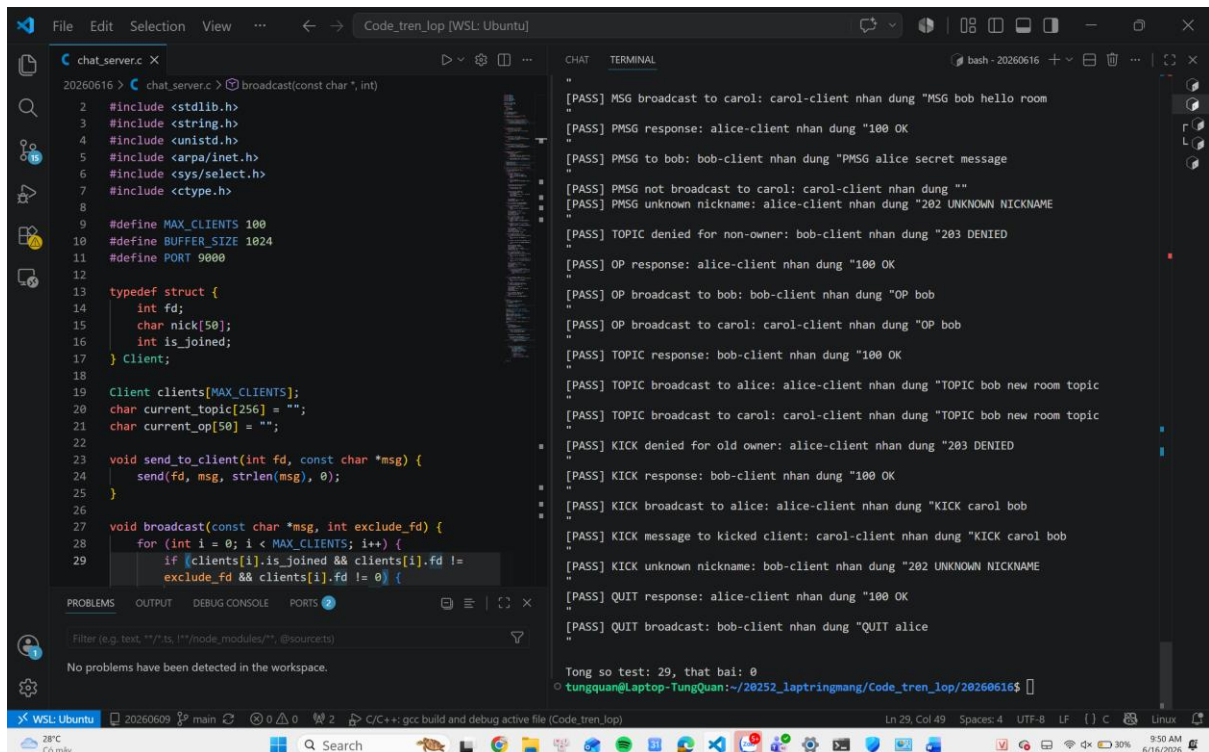


```
File Edit Selection View ... Code_tren_lop [WSL: Ubuntu]
chat_server.c X
20260616 > chat_server.c > _unnamed_struct_0df0_1 > fd
209 server_fd = socket(AF_INET, SOCK_STREAM, 0);
210 int opt = 1;
211 setsockopt(server_fd, SOL_SOCKET, SO_REUSEADDR, &opt, sizeof(opt));
212
213 address.sin_family = AF_INET;
214 address.sin_addr.s_addr = INADDR_ANY;
215 address.sin_port = htons(PORT);
216
217 bind(server_fd, (struct sockaddr *)&address, sizeof(address));
218 listen(server_fd, 10);
219
220 printf("Chat server is running on port %d...\n", PORT);
221
222 while (1) {
223     FD_ZERO(&readfds);
224     FD_SET(server_fd, &readfds);
225     max_sd = server_fd;
226
227     for (int i = 0; i < MAX_CLIENTS; i++) {
228         int sd = clients[i].fd;
229         if (sd > 0) FD_SET(sd, &readfds);
230         if (sd > max_sd) max_sd = sd;
231     }
232
233     select(max_sd + 1, &readfds, NULL, NULL, NULL);
234
235     if (FD_ISSET(server_fd, &readfds)) {
236         new_socket = accept(server_fd, NULL, NULL);
237         for (int i = 0; i < MAX_CLIENTS; i++) {
238             if (clients[i].fd == 0) {
239                 clients[i].fd = new_socket;
240                 break;
241             }
242         }
243     }
244 }
```



```
20260616 > chat_server.c > __unnamed_struct_0d0_1 > fd
241     }
242 }
243 }
244
245 for (int i = 0; i < MAX_CLIENTS; i++) {
246     int sd = clients[i].fd;
247     if (FD_ISSET(sd, &readfds)) {
248         char buffer[BUFFER_SIZE];
249         memset(buffer, 0, BUFFER_SIZE);
250         int valread = read(sd, buffer, BUFFER_SIZE - 1);
251
252         if (valread == 0) {
253             remove_client(i, 0);
254         } else {
255             handle_client_message(i, buffer);
256         }
257     }
258 }
259 }
260 return 0;
261 }
```

Kết quả:



```
20260616 > chat_server.c > broadcast(const char *, int)
2 #include <stdlib.h>
3 #include <string.h>
4 #include <unistd.h>
5 #include <arpa/inet.h>
6 #include <sys/select.h>
7 #include <ctype.h>
8
9 #define MAX_CLIENTS 100
10 #define BUFFER_SIZE 1024
11 #define PORT 9000
12
13 typedef struct {
14     int fd;
15     char nick[50];
16     int is_joined;
17 } Client;
18
19 Client clients[MAX_CLIENTS];
20 char current_topic[256] = "";
21 char current_op[50] = "";
22
23 void send_to_client(int fd, const char *msg) {
24     send(fd, msg, strlen(msg), 0);
25 }
26
27 void broadcast(const char *msg, int exclude_fd) {
28     for (int i = 0; i < MAX_CLIENTS; i++) {
29         if (clients[i].is_joined && clients[i].fd !=
            exclude_fd && clients[i].fd != 0) {
```

CHAT TERMINAL

```
bash - 20260616
[PASS] MSG broadcast to carol: carol-client nhan dung "MSG bob hello room"
[PASS] PMSG response: alice-client nhan dung "100 OK"
[PASS] PMSG to bob: bob-client nhan dung "PMSG alice secret message"
[PASS] PMSG not broadcast to carol: carol-client nhan dung ""
[PASS] PMSG unknown nickname: alice-client nhan dung "202 UNKNOWN NICKNAME"
[PASS] TOPIC denied for non-owner: bob-client nhan dung "203 DENIED"
[PASS] OP response: alice-client nhan dung "100 OK"
[PASS] OP broadcast to bob: bob-client nhan dung "OP bob"
[PASS] OP broadcast to carol: carol-client nhan dung "OP bob"
[PASS] TOPIC response: bob-client nhan dung "100 OK"
[PASS] TOPIC broadcast to alice: alice-client nhan dung "TOPIC bob new room topic"
[PASS] TOPIC broadcast to carol: carol-client nhan dung "TOPIC bob new room topic"
[PASS] KICK denied for old owner: alice-client nhan dung "203 DENIED"
[PASS] KICK response: bob-client nhan dung "100 OK"
[PASS] KICK broadcast to alice: alice-client nhan dung "KICK carol bob"
[PASS] KICK message to kicked client: carol-client nhan dung "KICK carol bob"
[PASS] KICK unknown nickname: bob-client nhan dung "202 UNKNOWN NICKNAME"
[PASS] QUIT response: alice-client nhan dung "100 OK"
[PASS] QUIT broadcast: bob-client nhan dung "QUIT alice"

Tong so test: 29, that bai: 0
tungquan@Laptop-TungQuan:~/20252_laptrimgang/Code_tren_lop/20260616$
```